INCIDENTREPORT

Simulated Ransomware Intrusion on ubuntu-victim

Home SOC Lab: Red Team Simulation and Blue Team Response

Field	Detail	Field	Detail
Report ID	HSL-IR-003	Severity	High (Wazuh level 12 alert)
Analyst	Senamile N.	Report date	8 October 2026
Incident window	1 Oct 12:26 to 2 Oct 09:23 (UTC)	Status	Recovered; remediation open (see 7.3)
Affected asset	ubuntu-victim, 192.168.56.20 (Wazuh agent 001)	Attacker host	192.168.56.1 (red team workstation)
Account used	jdoe (uid 1001)	Nature	Authorised lab exercise (simulated attack)

Executive summary

This report documents a controlled, multi-stage attack against a Linux endpoint in the Home SOC Lab and the detection and response that followed. The **red team** brute-forced SSH for the account `jdoe` on 1 October, logged in with a weak password, and on 2 October simulated ransomware that renamed four business documents to `.locked` and dropped a ransom note, all in about 64 milliseconds. The **blue team** followed the NIST incident response lifecycle: Wazuh, auditd and file integrity monitoring (FIM) detected every stage at least once, the custom rule 100041 raised a level 12 ransomware alert 43 ms after the first file was deleted, and the files were restored and verified at 09:22 UTC. No data was lost.

The review also found gaps: a custom rule that cannot tell an administrator's restore from ransomware (a false positive), a default brute-force threshold that left a 7-failure burst at level 5, and three of six successful logins that never reached Wazuh before a second log source was added, each followed within about a second by a journald rotation notice. Section 6 lists the fixes.

Why it matters. Had this been a production file server, one guessed password would have given an outsider read and write access to payroll and employee records, and the "encryption" would have finished in 64 ms, before any person could react. Wazuh saw every stage, but detection alone would not have saved the data. Recovery worked here only because the simulation renamed files; real ransomware overwrites file contents, and the files could then only come back from backups, which this lab does not have. The sections below therefore treat each gap in terms of what it would cost, and give steps another analyst can follow to close it.

84 s	64 ms	43 ms	4 / 4
first failure to level 10 escalation (rule 5763)	first delete to ransom note (attack duration)	first delete to level 12 alert (rule 100041)	files restored with original sizes and dates

1. Scope, environment and method

The lab runs on a host-only network (192.168.56.0/24). All activity was authorised, isolated and reversible. Times are **UTC** as shown in the terminal; the Wazuh dashboard displays local time (UTC+2), so the same alert appears two hours later there.

Host	Address	Role
wazuh-server	192.168.56.5	Wazuh manager, indexer and dashboard; stores alerts in /var/ossec/logs/alerts
ubuntu-victim	192.168.56.20	Monitored Ubuntu endpoint (agent 001) with auditd, realtime FIM on /home/jdoe/company_docs and SSH enabled
Red team host	192.168.56.1	Windows workstation running PowerShell and plink (PuTTY CLI) to drive the attack

How this report is organised. Part A (red team) describes what was done and why. Part B (blue team) is structured by the incident response lifecycle shown below, so each phase can be read on its own. Each blue team claim is tied to a screenshot or log extract.

PART A | RED TEAM

Adversary simulation: objectives, attack chain and evidence

2. Objectives and attack chain

Objective: test whether the SOC would detect a realistic credential-based intrusion that ends in ransomware-style impact. The attacker model is an outsider with network access to SSH on the endpoint and no prior credentials. Nothing destructive was used: "encryption" was a rename to `.locked`, which still triggers the same file-system signals as real ransomware.

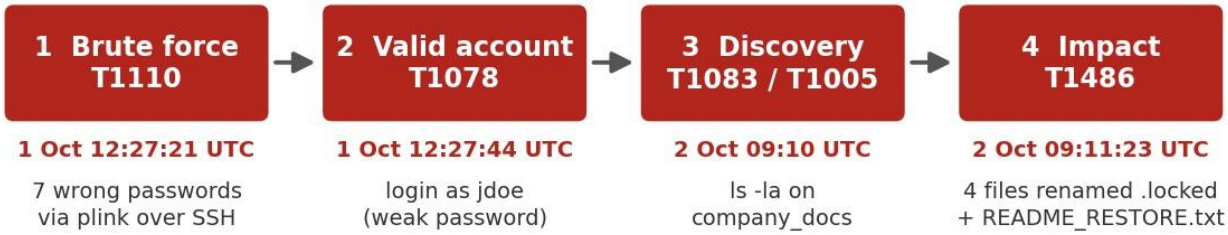


Figure 1. *Red team attack chain, mapped to MITRE ATT&CK (UTC).*

#	Tactic	Technique	Action performed	Time (UTC)
1	Credential Access	T1110 Brute Force	Seven wrong-password SSH logins as jdoe, two seconds apart, run from PowerShell with plink	1 Oct 12:27:21
2	Initial Access	T1078 Valid Accounts	Eighth attempt used the weak password `password123`; `whoami` returned jdoe	1 Oct 12:27:44
3	Discovery / Collection	T1083, T1005	Listed `/home/jdoe/company_docs` with `ls -la` to find files worth encrypting	2 Oct 09:10 to 09:11
4	Impact	T1486 Data Encrypted for Impact	Renamed Confidential.txt, Employee_Records.docx, Invoices.pdf and Payroll.xlsx to `.locked`; wrote README_RESTORE.txt	2 Oct 09:11:23

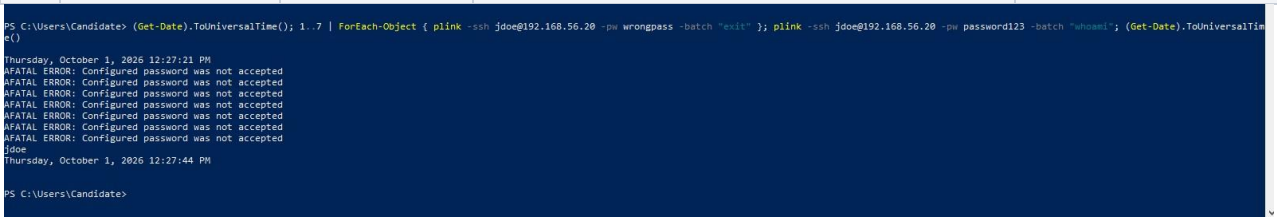


Figure 2. *PowerShell on the attacker host: seven rejected passwords, then a successful login (`whoami` returns jdoe). Clock reads 12:27:21 to 12:27:44 UTC, 1 Oct 2026.*

What the red team learned

- **Weak credentials were the entry point.** The password `password123` fell on the eighth attempt; no account lockout or rate limit slowed the attacker.
- **The attack was fast and noisy.** All activity came from one source (192.168.56.1) with no evasion, and the rename-based "encryption" of four files finished in 64 ms.
- **Every step left a trace** in sshd, PAM, auditd or file integrity logs, which is what the blue team used in Part B.

PART B | BLUE TEAM

Detection and response, structured by the incident response lifecycle (NIST SP 800-61)

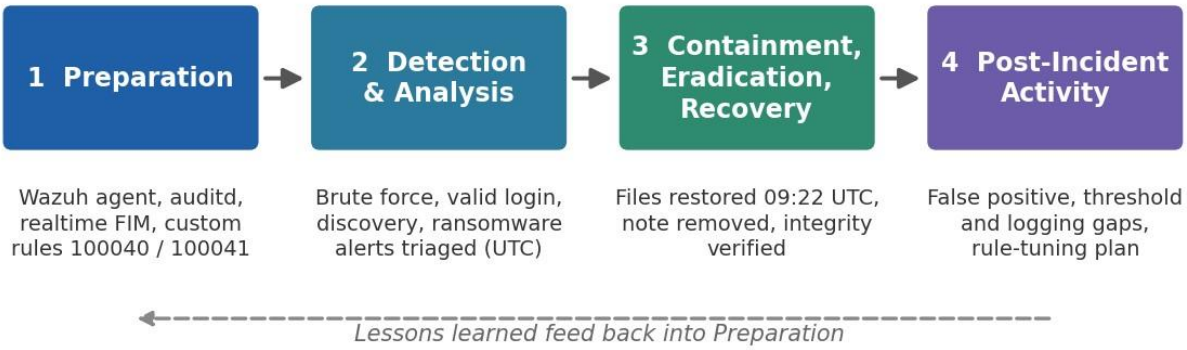


Figure 3. *Incident response lifecycle used for the blue team response, with what was done in each phase.*

3. Phase 1: Preparation

Preparation decided what the SOC could see before the attack began. The controls below were all in place before 1 October.

Control	Configuration	Why it matters
Wazuh agent	Agent 001 on ubuntu-victim reports to wazuh-server (192.168.56.5)	Central collection of SSH, PAM and audit events
auditd command logging	`-a always,exit -F arch=b64 -S execve -F auid>=1000 -F key=audit-wazuh-c`	Records every command a real user runs
File integrity monitoring	Realtime mode on `/home/jdoe/company_docs` (syscheck.mode = realtime)	Sees renames and deletes within milliseconds
Custom rule 100040	Level 9, child of 80792, matches `company_docs` in any audited command; T1083, T1005	Flags discovery or collection on the sensitive folder
Custom rule 100041	Level 12, frequency 3, timeframe 60 s, built on rule 553 (file deleted); T1486	Flags ransomware-style mass deletion

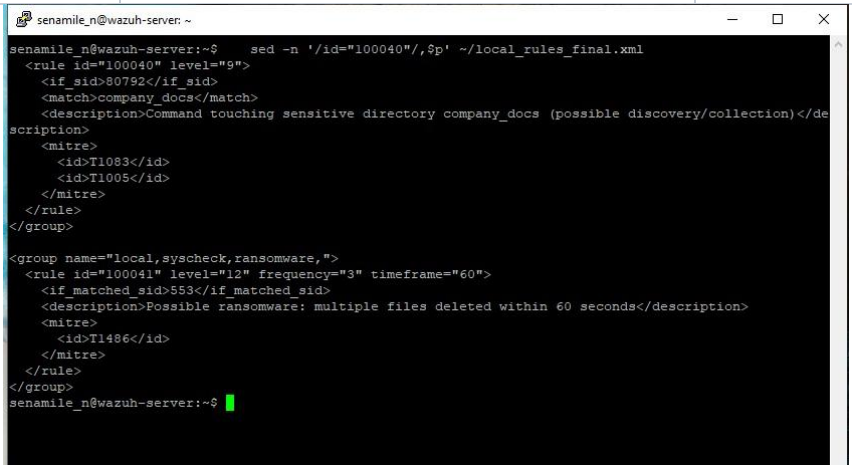


Figure 4. *Custom rules 100040 and 100041 in `local_rules_final.xml`.*

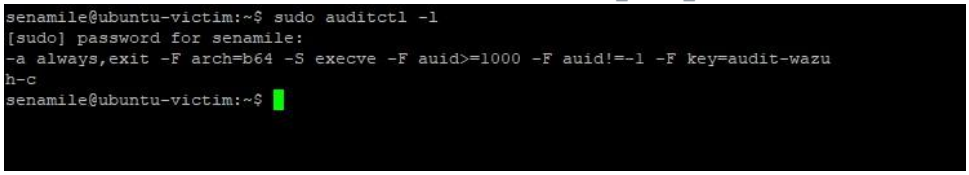


Figure 5. *Active audit rule (`auditctl -l`): all execve calls by real users carry the key audit-wazuh-c.*

4. Phase 2: Detection and analysis 4.1

Incident timeline (UTC)

Time (UTC)	Event	Rule / source
1 Oct 10:02:59 to 10:03:36	Test burst: 7 failures then a successful jdoe login. No escalation and no success alert (see 4.5)	5760, 5503 (L5)

1 Oct 12:26:01 to :17	First brute-force burst: 7 failed sshd logins from 192.168.56.1	5760, 5503 (L5)
1 Oct 12:27:23.983	PAM reports multiple failed logins in a short period	5551 (L10)
1 Oct 12:27:25.943	sshd brute force alert, 84 s after the first failure	5763 (L10), T1110
1 Oct 12:27:45.972	Multiple failures followed by a success: jdoe is compromised	40112 (L12), T1078
2 Oct 09:10:32 to :33	jdoe SSH login, then first audited command on company_docs	5715 (L3), 100040 (L9)
2 Oct 09:11:23.434	First file deleted (Confidential.txt)	553 (L7), FIM
2 Oct 09:11:23.477	**Ransomware alert**: three deletions within 60 s (trigger: Invoices.pdf)	**100041 (L12), T1486**
2 Oct 09:11:23.498	README_RESTORE.txt added: attack complete (64 ms after first delete)	554 (L5), FIM
2 Oct 09:11:24.133 to .142	Five audited commands touching company_docs	100040 (L9)
2 Oct 09:22:18 to :19.850	Administrator logs in and restores the files; rule 100041 fires again at 09:22:19.832	5715, 553, 554, 100041

4.2 Stages 1 and 2: brute force and compromised login

Wazuh's built-in ruleset covered this stage. The 1 October window produced **13** `5760` (sshd authentication failed) and **13** `5503` (PAM login failed) alerts at level 5, then one `5551` and one `5763` at level 10, and finally `40112` at level 12, which ties the failures to the successful login in one alert. The `5763` escalation arrived **84 seconds** after the first failure because it needs 8 failures from one source within 120 seconds; until then the activity sat at level 5.

What I make of this. The level 5 alerts (5760, 5503) mean very little on their own, because failed SSH logins are routine background noise on any host with SSH open. What changed my reading was the pattern: seven failures from one source, two seconds apart, is a script, not a person mistyping a password. 5551 and 5763 then confirm a brute force, but they only tell me that someone is trying. The alert that matters most is 40112, because it says an attempt succeeded. From that moment the question changes from "is someone attacking us?" to "what has this account done since?", so I would treat the incident as confirmed at 12:27:45 and start examining the jdoe session. There is also a timing detail that matters for the fixes in section 6: 5763 fired at 12:27:25.943, about 18 seconds before the successful login at 12:27:44. The attacker was already visible at level 10 before the password was guessed, so an automatic block on 5763 would have had time to stop this attack in this run, though with little margin.

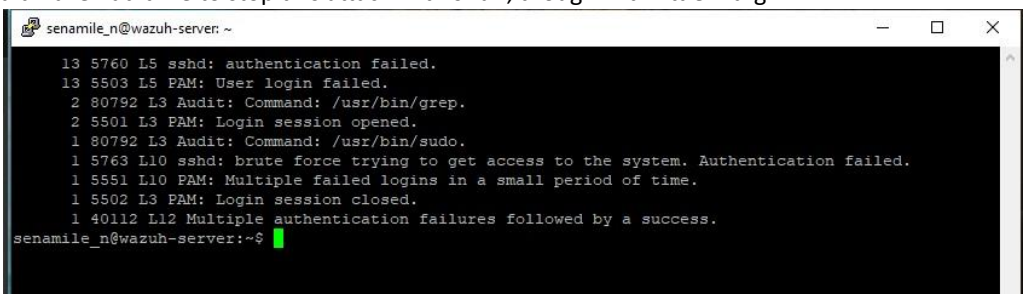


Figure 6. *Alert counts by rule for ubuntu-victim, 1 Oct 12:20 to 12:35 UTC. The two rule 80792 rows are the analyst's own `grep` and `sudo` commands, not attacker activity.*

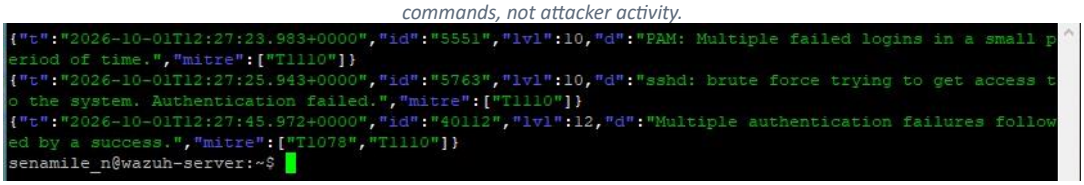


Figure 7. *Escalation alerts 5551, 5763 and level 12 rule 40112, with MITRE IDs (1 Oct, UTC).*

```
senamile_n@wazuh-server:~$ ^C
senamile_n@wazuh-server:~$ sudo jq -r 'select(.timestamp>="2026-10-01T12:25:50" and .
timestamp<"2026-10-01T12:28:00" and (.rule.id=="5760" or .rule.id=="5763" or .rule.id
=="40112")) | [.timestamp, .rule.id, .rule.level] | @tsv' /var/ossec/logs/alerts/2026
/Oct/ossec-alerts-01.json | tee ~/evidence-inv3-threshold-timeline.tsv
2026-10-01T12:26:01.915+0000    5760    5
2026-10-01T12:26:03.916+0000    5760    5
2026-10-01T12:26:05.920+0000    5760    5
2026-10-01T12:26:09.922+0000    5760    5
2026-10-01T12:26:13.923+0000    5760    5
2026-10-01T12:26:15.925+0000    5760    5
2026-10-01T12:26:17.925+0000    5760    5
2026-10-01T12:27:25.943+0000    5763    10
2026-10-01T12:27:31.948+0000    5760    5
2026-10-01T12:27:33.949+0000    5760    5
2026-10-01T12:27:35.954+0000    5760    5
2026-10-01T12:27:37.956+0000    5760    5
2026-10-01T12:27:41.957+0000    5760    5
2026-10-01T12:27:45.959+0000    5760    5
2026-10-01T12:27:45.972+0000    40112   12
senamile_n@wazuh-server:~$
```

Figure 8. *Threshold timeline: the first failure is at 12:26:01, but 5763 only fires at 12:27:25.943 and 40112 at 12:27:45.972.*

4.3 Stages 3 and 4: discovery and ransomware impact

Both custom rules fired. File integrity monitoring recorded each rename as a delete (`553`) followed by an add (`554`). The first delete and the last add are only **64 ms** apart, so the encryption step was finished before any person or script could react. Rule `100041` fired at 09:11:23.477, in the middle of the run, on the third deletion. Invoices.pdf shows no separate `553` row because that deletion was the one that triggered `100041`; the alert's `previous_output` field lists the other two deletions. Auditd command alerts (`100040`) arrived about 0.7 s later because audit records are read from the log after the commands complete.

What I make of this. The two custom rules answer different questions. 100041 says damage is happening, and it arrives too late to prevent anything. 100040 says someone is looking inside the sensitive folder straight after a login, and it fired about 50 seconds before the first file was renamed (09:10:33 compared with 09:11:23). That 50-second gap is the only point in the incident where a response could have changed the outcome. The catch is that 100040 also fires on any harmless `ls` by an administrator, so on its own it is too noisy to trigger isolation. I would want it correlated: 100040 from a source that recently failed many logins is a much stronger signal than either alone. The speed also tells me something: 64 ms for four files is far faster than a human could work, so this was scripted, and speed alone cannot separate this simulation from real ransomware.

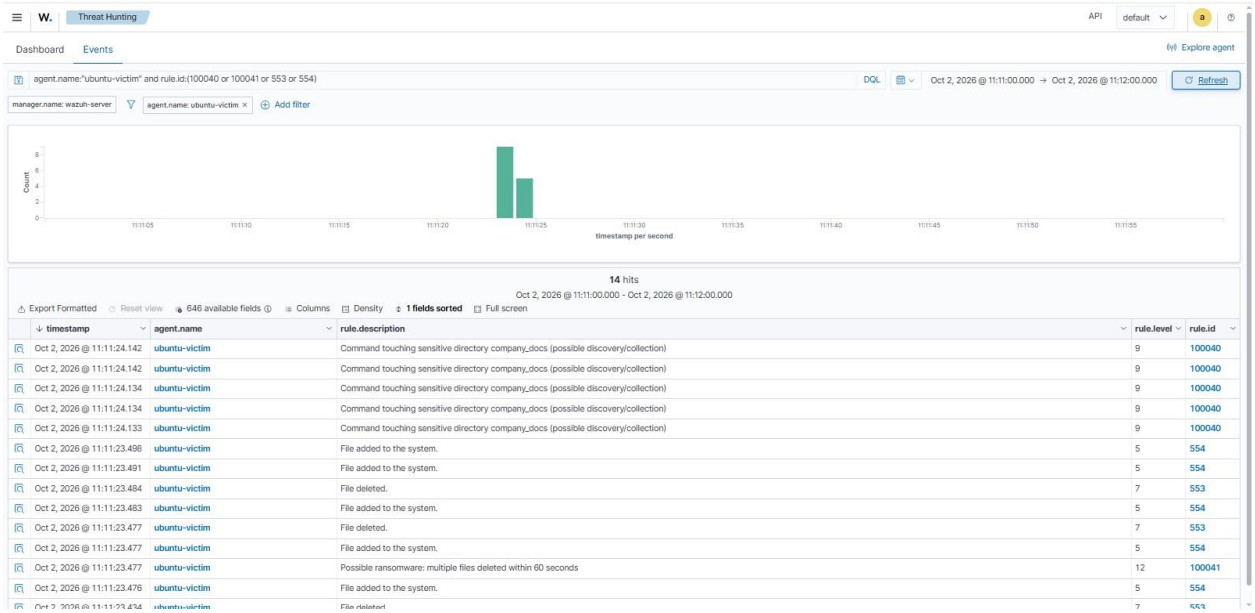


Figure 9. *Wazuh Threat Hunting view, 2 Oct 11:11:00 to 11:12:00 local (09:11 UTC): 14 hits, covering rules 100040, 100041, 553 and 554 on ubuntu-victim.*

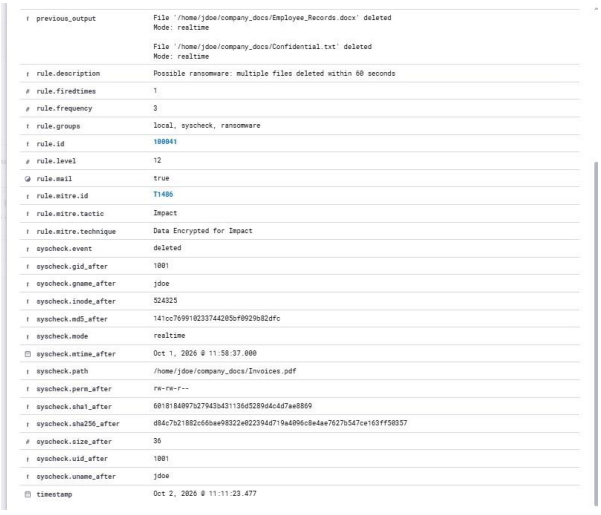


Figure 10. *Rule 100041 detail: level 12, frequency 3, T1486 (Impact), FIM mode realtime, rule.mail = true.*

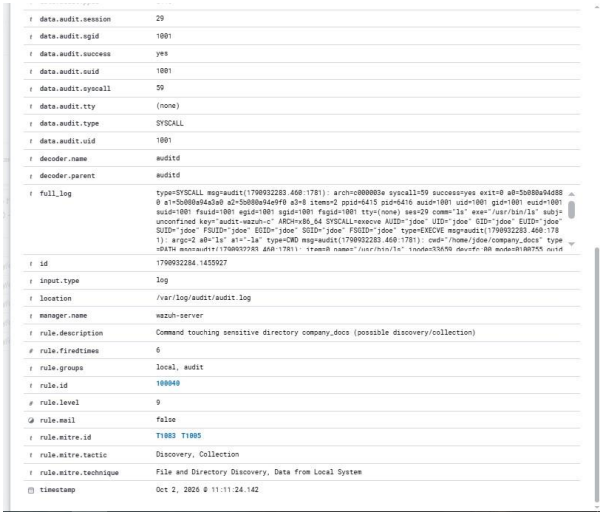


Figure 11. *Rule 100040 detail: `ls -la` in /home/jdoe/company_docs, level 9, T1083 and T1005.*

```
senamile_n@wazuh-server:~$ sudo jq -r 'select(.agent.name=="ubuntu-victim" and (.rule.id=="553" or .ru
le.id=="554") and .timestamp>="2026-10-02T09:11:00" and .timestamp<"2026-10-02T09:12:00") | [.timestam
p, .rule.id, .syscheck.event, .syscheck.path] | @tsv' /var/ossec/logs/alerts/2026/Oct/ossec-alerts-02.
json
[sudo] password for senamile_n:
2026-10-02T09:11:23.434+0000    553     deleted /home/jdoe/company_docs/Confidential.txt
2026-10-02T09:11:23.476+0000    554     added   /home/jdoe/company_docs/Confidential.txt.locked
2026-10-02T09:11:23.477+0000    553     deleted /home/jdoe/company_docs/Employee_Records.docx
2026-10-02T09:11:23.477+0000    554     added   /home/jdoe/company_docs/Employee_Records.docx.locked
2026-10-02T09:11:23.483+0000    554     added   /home/jdoe/company_docs/Invoices.pdf.locked
2026-10-02T09:11:23.484+0000    553     deleted /home/jdoe/company_docs/Payroll.xlsx
2026-10-02T09:11:23.491+0000    554     added   /home/jdoe/company_docs/Payroll.xlsx.locked
2026-10-02T09:11:23.498+0000    554     added   /home/jdoe/company_docs/README_RESTORE.txt
senamile_n@wazuh-server:~$
```

Figure 12. *FIM events from alerts-02.json (UTC): four deletes, four adds, ending with the ransom note.*


```
er":"jdoe"}
{"t":"2026-10-02T09:11:24.162+0000","id":"5501","lvl":3,"d":"PAM: Login session opened.","ip":null,"us
er":"jdoe"}
{"t":"2026-10-02T09:11:24.552+0000","id":"5715","lvl":3,"d":"sshd: authentication success.","ip":"192.
168.56.1","user":"jdoe"}
{"t":"2026-10-02T09:11:24.557+0000","id":"5501","lvl":3,"d":"PAM: Login session opened.","ip":null,"us
er":"jdoe"}
{"t":"2026-10-02T09:11:24.586+0000","id":"5501","lvl":3,"d":"PAM: Login session opened.","ip":null,"us
er":"jdoe"}
{"t":"2026-10-02T09:22:18.823+0000","id":"5715","lvl":3,"d":"sshd: authentication success.","ip":"192.
168.56.1","user":"jdoe"}
{"t":"2026-10-02T09:22:18.823+0000","id":"5501","lvl":3,"d":"PAM: Login session opened.","ip":null,"us
er":"jdoe"}
{"t":"2026-10-02T09:22:18.852+0000","id":"5501","lvl":3,"d":"PAM: Login session opened.","ip":null,"us
er":"jdoe"}
{"t":"2026-10-02T09:22:18.944+0000","id":"5715","lvl":3,"d":"sshd: authentication success.","ip":"192.
168.56.1","user":"jdoe"}
{"t":"2026-10-02T09:22:18.944+0000","id":"5501","lvl":3,"d":"PAM: Login session opened.","ip":null,"us
er":"jdoe"}
{"t":"2026-10-02T09:22:18.944+0000","id":"5501","lvl":3,"d":"PAM: Login session opened.","ip":null,"us
er":"jdoe"}
senamile_n@wazuh-server:~$
```


Figure 13. *sshd alerts on wazuh-server: successful jdoe logins from 192.168.56.1 at 09:06, 09:10 and 09:11 (attack session) and 09:22 UTC (administrator restore session).*

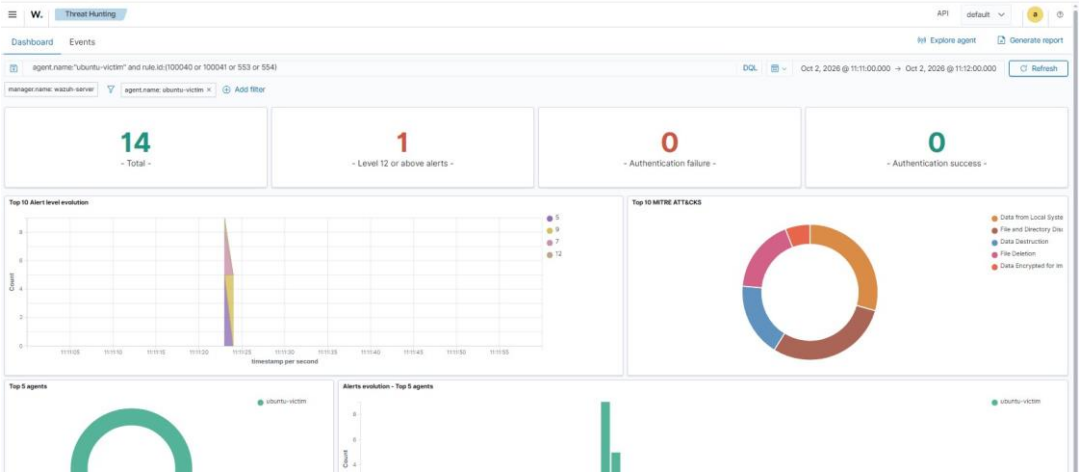


Figure 14. *Dashboard for the attack minute: 14 alerts, 1 at level 12 or above, MITRE techniques Data Encrypted for Impact, File Deletion, Data Destruction and Discovery. ubuntu-victim is the only agent affected.*

4.4 Detection coverage: red actions against blue detections

Red team action	MITRE	Wazuh rule	Result	Notes

SSH brute force (1 Oct 12:26)	T1110	5760, 5503, 5551, 5763	Detected	Built-in; 84 s to level 10
Login after failures	T1078	40112 (L12)	Detected	Built-in; strongest signal in stage 1
Login after 7 failures (1 Oct 10:03)	T1078	5760, 5503 only	**Missed**	Below threshold; success event lost
Discovery in company_docs	T1083, T1005	100040 (L9)	Detected	Custom rule
Rename to .locked	T1486	100041 (L12)	Detected	Custom rule; 43 ms after first delete

4.5 Detection gap: the 1 October 10:03 login

An earlier test on 1 Oct produced 7 failed logins (10:02:59 to 10:03:33) followed by a successful jdoe login at 10:03:36. Wazuh alerted on the failures only at level 5, and **no alert exists for the successful login**, although journalctl shows it. Two causes were checked:

- **Agent restart or disconnect: ruled out.** ossec.log shows the agent started at 09:46:35 and lost its manager connection only at 10:22:23, so it was connected and running at 10:03.
- **Journald rotation: working hypothesis, not proven.** The logcollector logged "Journald timestamp was refreshed due to rotation" at 10:03:37, one second after the login, so my working explanation is that the success event was skipped during the rotation. The table below separates what the evidence proves from what I infer.

Even if the event had arrived, 7 failures stay below the 8-in-120-second threshold, so rule 40112 might not have linked the success to the failures. The gap therefore has two parts: a logging loss and a detection threshold (finding 6.2).

Later evidence shows this was not a one-off. Comparing every successful SSH login in the journal with what reached Wazuh, every login followed by a rotation notice within one second lost its journald copy, while the logins followed after two seconds, or with no notice, were delivered normally. Before /var/log/auth.log was added on 2 Oct, three logins were lost entirely. Afterwards, the two logins that journald lost on 8 Oct were still delivered through auth.log.

Successful SSH login (UTC)	Journald rotation notice	Reached Wazuh?
1 Oct 09:48:25 (senamile)	09:48:27 (+2 s)	Yes (5715)
1 Oct 10:03:36 (jdoe)	10:03:37 (+1 s)	No
1 Oct 10:27:04 (jdoe)	none	Yes (5715)
1 Oct 12:27:44 (jdoe)	none	Yes (40112)
2 Oct 07:19:46 (senamile)	07:19:47 (+1 s)	No
2 Oct 07:30:51 (jdoe)	07:30:52 (+1 s)	No
2 Oct 07:56:40 (jdoe), after the fix	none	Yes, both sources (5715)
8 Oct 07:44:10 (senamile), after reboot	07:44:11 (+1 s)	Yes, auth.log only (5715)
8 Oct 07:46:37 (jdoe)	07:46:39 (+2 s)	Yes, both sources (5715)
8 Oct 07:50:04 (jdoe)	none	Yes, both sources (5715)
8 Oct 07:51:46 (senamile)	07:51:47 (+1 s)	Yes, auth.log only (5715)

The pattern suggests a timing race between the login and the rotation, but the cause is not proven. The fix, however, is confirmed. After /var/log/auth.log was added on 2 Oct, the victim was rebooted on 8 Oct, and the two logins that journald lost (07:44:10 and 07:51:46) each reached Wazuh through auth.log alone (rule 5715). Logins that journald does deliver now alert from both sources, so duplicate alerts are expected.

What I proved and what I believe

The conclusion is only as strong as its weakest link, so each statement is labelled by how it is known.

Statement	Status	Basis
The 1 Oct 10:03:36 jdoe login happened	**Proved**	journalctl shows `Accepted password for jdoe` (Figure 16)
Statement	**Status**	**Basis**

No 5715 alert for that login reached Wazuh	**Proved**	alerts.json for 10:02:59 to 10:03:39 has no 5715 (Figure 15)
The agent was running and connected at the time	**Proved**	ossec.log: started 09:46:35, lost connection only at 10:22:23 (Figure 17)
auth.log delivered both logins journald lost on 8 Oct	**Proved**	Figure 19: 07:44:11 and 07:51:46 arrive from `/var/log/auth.log` only
Lost logins are each user's first login after the agent starts	**Rejected**	The 07:51:46 login was senamile's second after the agent started, and it was lost too
Every login followed by a rotation notice 1 s later lost its journald copy	**Observed**	5 of 5 such logins lost; the 2 followed 2 s later were delivered (table above). Small sample
Adding auth.log closes the gap for SSH logins	**Supported**	Two recovered logins on 8 Oct and none lost since. Works whatever the cause is, but only two cases
Journald rotation causes the loss	**Not proven**	Correlation only. I have not run a controlled test or read logcollector debug output
The loss is a timing race with a window of about 1 second	**Belief**	Fits the 1 s versus 2 s split, but with 7 data points the boundary could be chance

How I would prove or disprove it. I have not run this test yet. Enable logcollector debug on ubuntu-victim by adding `logcollector.debug=2` to `/var/ossec/etc/local_internal_options.conf` and restarting the agent. Then force a journald rotation with `sudo journalctl --rotate`, log in over SSH at about 0 to 1 s, 2 to 3 s and 5 s or more after it, and repeat at least 20 times per delay. Compare the logins in journalctl with the 5715 alerts delivered from journald. If the losses appear only in the shortest delay, the timing-race hypothesis is supported. If they appear at every delay, or the debug log shows the event was never read, the explanation is wrong and the cause lies elsewhere.

Why it matters either way. The practical risk does not depend on the cause. The successful login is the event that rule 40112 and every post-compromise investigation start from, and losing it raises no error. A real brute force that succeeded during a rotation would look like a few level 5 failures and nothing else, which is exactly what the 10:03 test produced.

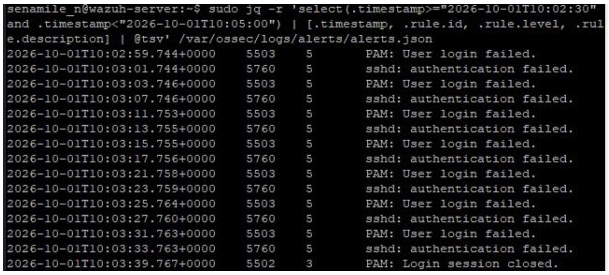


Figure 15. *Wazuh alerts 10:02:59 to 10:03:39: failures and a session-closed event, but no 5715 success at 10:03:36.*

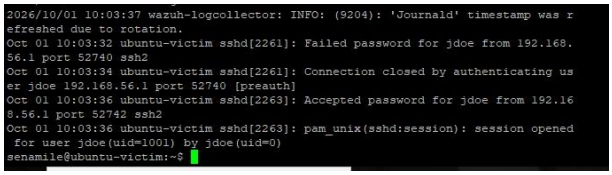


Figure 16. *journalctl shows "Accepted password for jdoe" at 10:03:36, right after the rotation notice.*

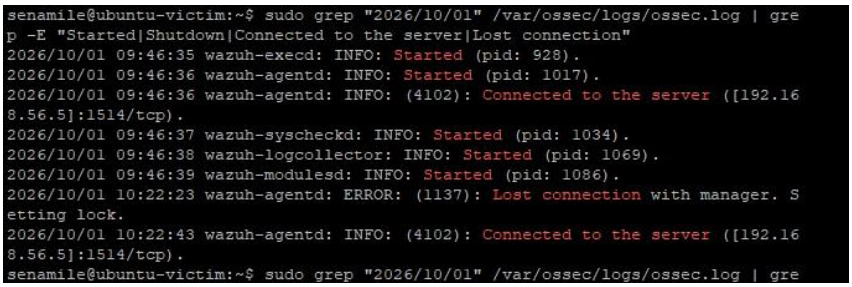


Figure 17. *ossec.log on ubuntu-victim: agent started 09:46:35 and lost connection only at 10:22:23, so no restart explains the 10:03 gap.*

```
Last login: Thu Oct  8 07:44:12 2026 from 192.168.56.1
senamile@ubuntu-victim:~$ sudo journalctl -u ssh --since "2026-10-08 07:30" --no
-pager | grep "Accepted password"
sudo grep "2026/10/08" /var/ossec/logs/ossec.log | grep -i rotation
[sudo] password for senamile:
Oct 08 07:44:10 ubuntu-victim sshd[1631]: Accepted password for senamile from 19
2.168.56.1 port 54706 ssh2
Oct 08 07:46:37 ubuntu-victim sshd[1903]: Accepted password for jdoe from 192.16
8.56.1 port 54740 ssh2
Oct 08 07:50:04 ubuntu-victim sshd[2029]: Accepted password for jdoe from 192.16
8.56.1 port 54779 ssh2
Oct 08 07:51:46 ubuntu-victim sshd[2124]: Accepted password for senamile from 19
2.168.56.1 port 54800 ssh2
2026/10/08 07:44:11 wazuh-logcollector: INFO: (9204): 'Journald' timestamp was r
efreshed due to rotation.
2026/10/08 07:46:39 wazuh-logcollector: INFO: (9204): 'Journald' timestamp was r
efreshed due to rotation.
2026/10/08 07:51:47 wazuh-logcollector: INFO: (9204): 'Journald' timestamp was r
efreshed due to rotation.
senamile@ubuntu-victim:~$
```

Figure 18. *journalctl and ossec.log on ubuntu-victim after the 8 Oct reboot: the logins at 07:44:10 and 07:51:46 are each followed by a journald rotation notice one second later (07:44:11, 07:51:47).*

```
senamile_n@wazuh-server:~$ sudo jq -r 'select(.agent.name=="ubuntu-victim" and (
(.full_log // "")|contains("Accepted password")) and .timestamp>="2026-10-08") |
 [.timestamp, .rule.id, .location] | @tsv' /var/ossec/logs/alerts/alerts.json |
tee ~/evidence-inv3-reboot-test.tsv
2026-10-08T07:44:11.194+0000    5715    /var/log/auth.log
2026-10-08T07:46:37.269+0000    5715    journald
2026-10-08T07:46:37.352+0000    5715    /var/log/auth.log
2026-10-08T07:50:04.695+0000    5715    /var/log/auth.log
2026-10-08T07:50:05.324+0000    5715    journald
2026-10-08T07:51:46.767+0000    5715    /var/log/auth.log
senamile_n@wazuh-server:~$
```

Figure 19. *Wazuh alerts for the same logins, saved as evidence-inv3-reboot-test.tsv: 07:44:11 and 07:51:46 arrive through /var/log/auth.log only, so auth.log delivered both logins that journald lost.*

5. Phase 3: Containment, eradication and recovery

Containment. The attack lasted 64 ms and the level 12 alert arrived 43 ms into it, so by the time I saw the alert the encryption was already finished, and no analyst or script could have stopped it. No automated response was configured, so I limited containment to preserving the evidence and making no further changes on the host. In a real deployment this timing means containment must be automated (see finding 6.4).

Eradication. I removed the `.locked` names and the ransom note `README_RESTORE.txt` as part of the restore. I found no malware binary, scheduled task or other persistence at this stage. I identified the root cause as the weak jdoe password, so the password still needs to be reset and SSH password logins restricted (finding 6.5).

Recovery. At 09:22 UTC I logged in as jdoe and moved the four files back to their original names. I then checked the restore against the evidence: all four files carry their original 1 Oct 09:58 timestamps, and `Invoices.pdf` is 36 bytes, matching `size_after` in the rule 100041 alert.

```
senamile@ubuntu-victim:~$    sudo ls -la /home/jdoe/company_docs
[sudo] password for senamile:
total 24
drwxrwxr-x 2 jdoe jdoe 4096 Oct  2 09:22 .
drwxr-x--- 4 jdoe jdoe 4096 Oct  1 10:03 ..
-rw-rw-r-- 1 jdoe jdoe   40 Oct  1 09:58 Confidential.txt
-rw-rw-r-- 1 jdoe jdoe   32 Oct  1 09:58 Employee_Records.docx
-rw-rw-r-- 1 jdoe jdoe   36 Oct  1 09:58 Invoices.pdf
-rw-rw-r-- 1 jdoe jdoe   40 Oct  1 09:58 Payroll.xlsx
senamile@ubuntu-victim:~$
```

Figure 20. *company_docs after recovery: four original filenames, 1 Oct 09:58 timestamps, folder modified 2 Oct 09:22.*

```
senamile_n@wazuh-server:~$ sudo jq -r 'select(.rule.id=="553" and .timestamp>="2026-10-02T09:22:00" an
d .timestamp<"2026-10-02T09:23:00") | [.timestamp,.syscheck.path] | @tsv' /var/ossec/logs/alerts/2026/
Oct/ossec-alerts-02.json
2026-10-02T09:22:19.791+0000    /home/jdoe/company_docs/Confidential.txt.locked
2026-10-02T09:22:19.823+0000    /home/jdoe/company_docs/Employee_Records.docx.locked
2026-10-02T09:22:19.849+0000    /home/jdoe/company_docs/Payroll.xlsx.locked
2026-10-02T09:22:19.850+0000    /home/jdoe/company_docs/README_RESTORE.txt
senamile_n@wazuh-server:~$
```

Figure 21. *The restore deleted the .locked files and the ransom note (09:22:19.791 to .850 UTC).*

62 hits

Oct 1, 2026 @ 14:33:13.548 - Oct 2, 2026 @ 14:33:13.548

Export Formatted | Reset view | 646 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Oct 2, 2026 @ 11:22:20.453	ubuntu-victim	Command touching sensitive directory company_docs (possible discovery/collection)	9	100040
Oct 2, 2026 @ 11:22:20.452	ubuntu-victim	Command touching sensitive directory company_docs (possible discovery/collection)	9	100040
Oct 2, 2026 @ 11:22:20.452	ubuntu-victim	Command touching sensitive directory company_docs (possible discovery/collection)	9	100040
Oct 2, 2026 @ 11:22:20.445	ubuntu-victim	Command touching sensitive directory company_docs (possible discovery/collection)	9	100040
Oct 2, 2026 @ 11:22:20.443	ubuntu-victim	Command touching sensitive directory company_docs (possible discovery/collection)	9	100040
Oct 2, 2026 @ 11:22:20.441	ubuntu-victim	Command touching sensitive directory company_docs (possible discovery/collection)	9	100040
Oct 2, 2026 @ 11:22:19.850	ubuntu-victim	File deleted.	7	553
Oct 2, 2026 @ 11:22:19.849	ubuntu-victim	File deleted.	7	553
Oct 2, 2026 @ 11:22:19.838	ubuntu-victim	File added to the system.	5	554
Oct 2, 2026 @ 11:22:19.832	ubuntu-victim	Possible ransomware: multiple files deleted within 60 seconds	12	100041
Oct 2, 2026 @ 11:22:19.830	ubuntu-victim	File added to the system.	5	554
Oct 2, 2026 @ 11:22:19.823	ubuntu-victim	File deleted.	7	553
Oct 2, 2026 @ 11:22:19.822	ubuntu-victim	File added to the system.	5	554
Oct 2, 2026 @ 11:22:19.809	ubuntu-victim	File added to the system.	5	554
Oct 2, 2026 @ 11:22:19.791	ubuntu-victim	File deleted.	7	553

Figure 22. *Dashboard, 2 Oct 11:22 local (09:22 UTC): the restore raised a second rule 100041 alert at 11:22:19.832. This is a false positive caused by the administrator's own action.*

6. Phase 4: Post-incident activity

The response worked: every stage was detected at least once and the data was fully recovered. The review found five weaknesses, listed below with their priority.

#	Finding	Evidence	Recommended action
6.1	**Rule 100041 cannot tell restore from ransomware (High)**	Fired at 09:11:23.477 (attack) and 09:22:19.832 (restore); both are 3+ deletes in 60 s in one folder	Count only deletes paired with `.locked` adds (rule 554), raise the count, or use FIM whodata to see the process responsible
6.2	**Brute-force threshold leaves bursts at level 5 (Medium)**	5763 needs 8 failures in 120 s; a 7-failure burst stayed at level 5 and took 84 s to escalate on 1 Oct	Add a custom rule at 5 failures, and alert on any success after 3+ failures from one IP
6.3	**Login success lost to journald rotation (Medium)**	Before the fix, three of six successful logins (1 Oct 10:03:36; 2 Oct 07:19:46 and 07:30:51) were in journalctl but raised no alert; the agent was running, and logcollector logged a journald rotation about a second after each (see 4.5)	Applied 2 Oct: added /var/log/auth.log as a second syslog source on ubuntu-victim (agent restarted 07:55). Confirmed 8 Oct: after a reboot, the two logins that journald lost (07:44:10 and 07:51:46) reached Wazuh through auth.log alone (rule 5715). Logins journald does deliver now alert twice, so duplicates are expected; still recommended: keep a file source such as /var/log/auth.log as the primary SSH path, and alert when logcollector reports a rotation
6.4	**Detection is faster than any manual response (High)**	Alert at +43 ms, attack finished at +64 ms	Enable Wazuh active response: block the source IP on rule 5763, lock the account on 100041 (who-data), and keep a backup (steps below)
6.5	**Weak password accepted for jdoe (High)**	The password `password123` was accepted on the eighth attempt (1 Oct 12:27:44); no lockout stopped the brute force	Reset the password, enforce SSH key authentication, and add lockout or rate limiting (for example fail2ban)

Impact and how to fix each finding

The table ranks each gap by what it would cost in a real environment. The steps below it are written so that another analyst can follow them in order. The rule IDs and snippets are suggestions: I have not yet run them, so each one ends with a check to confirm it works before it is relied on. Back up the file before editing it, and run `sudo /var/ossec/bin/wazuh-analysisd -t` to check the rules before restarting the manager.

Finding	Security impact	Business impact	Effort
6.5 Weak password	Any guessed credential gives the attacker everything that account can read or change	Employee and payroll records exposed: a personal-data breach if this were a real company	Low
Finding	**Security impact**	**Business impact**	**Effort**
6.4 No automatic response	Encryption completes (64 ms) before any person can act	Files lost or unavailable; recovery depends on backups, which are not in place	Medium
6.3 Lost login events	A successful compromise can look like level 5 noise, with no error raised	Breach goes unnoticed, so it is found late or by the attacker's ransom note	Low (partly done)
6.2 Brute-force threshold	About 7 guesses per 2 minutes (roughly 5,000 a day, in theory) from one source stays below level 10	Slow guessing against weak passwords succeeds without anyone being paged	Low
6.1 False positive	A level 12 alert that fires on legitimate work lowers trust, and automation on it would act on the wrong event	Alert fatigue, and an automatic response could take the host or account offline during a legitimate restore	Medium

6.5 Weak password accepted for jdoe

Why it matters. This is the root cause. One guessed password gave the attacker everything jdoe could read, including the folder with payroll and employee records, and every other finding is a safety net around this hole. Closing it is cheap and removes the attack path entirely.

Steps.

- **Step 1.** Reset the password on ubuntu-victim with `sudo passwd jdoe`, using a long passphrase.
- **Step 2.** On the Windows admin host, create a key with `ssh-keygen -t ed25519`, then install it on the victim: `type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh jdoe@192.168.56.20 "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"`. On the victim, set permissions with `chmod 700 ~/.ssh` and `chmod 600 ~/.ssh/authorized_keys`.
- **Step 3.** Open a second terminal and confirm that key login works before changing anything else. Keep the first session open until the end.
- **Step 4.** In `/etc/ssh/sshd_config` set `PasswordAuthentication no`. Also check `/etc/ssh/sshd_config.d/` for a file that sets it back to `yes`, which some Ubuntu images do. Then run `sudo sshd -t && sudo systemctl restart ssh`.
- **Step 5.** Add rate limiting with fail2ban: `sudo apt install fail2ban`, then put the lines below in `/etc/fail2ban/jail.local` and run `sudo systemctl restart fail2ban`.

```
[sshd] enabled
= true
maxretry = 5
findtime = 10m
bantime = 1h
```

Check. Re-run the PowerShell brute force. Every attempt should be refused immediately with `Permission denied (publickey)` and rule 40112 should not fire. Note that in this lab the attacker and the administrator share 192.168.56.1, so fail2ban will ban the administrator's address during the test; use the VirtualBox console to get back in, or run `sudo fail2ban-client unban 192.168.56.1`.

6.4 Detection is faster than any manual response

Why it matters. The alert arrived at +43 ms and the attack ended at +64 ms, so only an automatic response can change the outcome. Two practical points shape the fix. First, the built-in `firewall-drop` response needs a source IP, and file integrity alerts such as 100041 do not carry one, so it cannot be tied to 100041 directly. Second, 5763 does carry the attacker's IP and fired about 18 seconds before the password was guessed, so blocking on it is the realistic first step. Finally, the lab recovered only because the files were renamed. Without backups, real ransomware would mean permanent loss.

Steps.

- **Step 1.** On wazuh-server, add the block below to `/var/ossec/etc/ossec.conf` inside `<ossec_config>`, so that repeated SSH failures get the source IP blocked on the victim for 10 minutes. Rule 100050 is created in finding 6.2, so do that step first or list only 5763 for now.

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763,100050</rules_id>
  <timeout>600</timeout> </active-
response>
```

- **Step 2.** Run `sudo systemctl restart wazuh-manager`. Do not use a whitelist for 192.168.56.1 while testing, because that is the attacker's address in this lab.
- **Step 3.** For ransomware, switch FIM on `/home/jdoe/company_docs` to who-data (`whodata="yes"` in the `<directories>` line of the victim's ossec.conf; it relies on auditd, which is already running) so the alert names the user. Then write a small active-response script that locks that account, and attach it to the ransomware rule. Test it on a spare account first.
- **Step 4.** Keep a backup the victim cannot overwrite. As root, run a nightly job such as `tar -czf /backup/company_docs-$(date +%F).tar.gz -C /home/jdoe company_docs`, store it where jdoe has no write access, and practise one restore.

Check. Re-run the brute force from the admin host. `sudo iptables -L -n` on the victim should show a DROP rule for the attacker IP, and `/var/ossec/logs/active-responses.log` should record it. Delete a test backup file and restore it to confirm the backup works.

6.3 Login success lost to journald rotation

Why it matters. The successful login is the event that turns an attack into a confirmed compromise. Rule 40112 depends on it, and any review of what an account did starts from it. Before the fix, 3 of 6 logins went missing with no error, so a real compromise could have sat unseen behind level 5 failures. The second log source covers the gap, but nothing yet tells an analyst if a source goes quiet again.

Steps.

- **Step 1.** Confirm the fix is still in place on ubuntu-victim with `grep -B1 -A2 auth.log /var/ossec/etc/ossec.conf`. It should show a `<localfile>` block with `<log_format>syslog</log_format>` and `<location>/var/log/auth.log</location>`.
- **Step 2.** Make rotation visible. Add `/var/ossec/logs/ossec.log` as a second `<localfile>` (same format) on the victim, then add the rule below on the server. I have not checked that ossec.log lines decode cleanly this way, so confirm with `wazuh-logtest`.

```
<rule id="100060" level="6">
  <match>timestamp was refreshed due to rotation</match>
  <description>Logcollector: journald rotation, SSH logins nearby may be missing from
journald</description>
</rule>
```

- **Step 3.** Add a weekly reconciliation. On the victim run `sudo journalctl -u ssh --since "7 days ago" --no-pager | grep -c "Accepted password"`. On the server run `sudo jq -r 'select(.agent.name=="ubuntu-victim" and .rule.id=="5715") | .timestamp' /var/ossec/logs/alerts/alerts.json | wc -l`. Because logins that journald delivers now alert twice, the alert count should be equal to or higher than the login count; a lower count means logins are being lost.

Check. Repeat the reboot test from 8 Oct and confirm that each journald rotation notice produces a 100060 alert and that the reconciliation counts agree.

6.2 Brute-force threshold leaves bursts at level 5

Why it matters. Rule 5763 needs 8 failures in 120 seconds. An attacker who stays under that, at about 7 per window, makes roughly 5,000 guesses a day from one address without ever reaching level 10, and level 5 alerts are not normally triaged. That is a theoretical rate based on the rule as I observed it, not something I tested. Against a weak password it means a slow attack can succeed without anyone being paged.

Steps.

- **Step 1.** Add the two rules below to the local rules file on wazuh-server. The first raises 5 failures from one source to level 10; the second raises a success after those failures to level 12, in the same way that 40112 does.

```
<rule id="100050" level="10" frequency="5" timeframe="120">
  <if_matched_sid>5760</if_matched_sid>
  <same_source_ip />
  <description>sshd: 5 failed logins from one source in 2 minutes</description>
  <mitre><id>T1110</id></mitre>
</rule>
<rule id="100051" level="12" timeframe="300">
  <if_group>authentication_success</if_group>
  <if_matched_sid>100050</if_matched_sid>
  <same_source_ip />
  <description>sshd: successful login after repeated failures from one source</description>
  <mitre><id>T1078</id></mitre>
</rule>
```

- **Step 2.** Test without touching the victim. Run `sudo /var/ossec/bin/wazuh-logtest`, then paste this line five times in one session and watch 100050 fire on the fifth: `Oct 8 07:00:01 ubuntu-victim sshd[1234]: Failed password for jdoe from 192.168.56.1 port 50000 ssh2`.
- **Step 3.** Run `sudo /var/ossec/bin/wazuh-analysisd -t`, then `sudo systemctl restart wazuhmanager`.

Check. Repeat the 1 Oct 10:03 test (7 failures then a success). It should now produce a level 10 alert and, if the success event arrives, a level 12 alert.

6.1 Rule 100041 cannot tell restore from ransomware

Why it matters. A level 12 alert that also fires on a legitimate restore teaches analysts to distrust level 12. It also becomes dangerous once finding 6.4 is fixed: an automatic response on 100041 would isolate the host or lock the account in the middle of a genuine recovery, so the response itself would cause an outage. The rule also only counts deletions, so an attacker who deletes fewer than three files a minute would never trigger it.

Steps.

- **Step 1.** Back up the rules file, for example `sudo cp /var/ossec/etc/rules/local_rules.xml ~/local_rules.bak`.
- **Step 2.** Add the two rules below. The first marks a file created with the `.locked` extension (the signature of this simulation); the second fires when three of those appear within 60 seconds. A restore deletes `.locked` files and recreates the originals, so it does not match.

```
<rule id="100042" level="8">
  <if_sid>554</if_sid>
  <field name="file" type="pcre2">\.locked$</field>
  <description>File created with .locked extension</description>
</rule>
<rule id="100043" level="12" frequency="3" timeframe="60">
  <if_matched_sid>100042</if_matched_sid>
  <description>Possible ransomware: 3 files renamed to .locked within 60 s</description>
  <mitre><id>T1486</id></mitre>
</rule>
```

- **Step 3.** Run `sudo /var/ossec/bin/wazuh-analysisd -t` and restart the manager. Once the new rule is proven, lower 100041 to level 8 rather than deleting it, so the original detection stays on record.

Check. Repeat the rename attack: 100043 should fire. Repeat the restore: no level 12 alert should appear. If the real attacker used a different extension, this rule would miss it, so keep a mass-deletion rule as a lower-level backup. That is why 100041 is kept rather than removed.

Evidence retained: twelve `evidence-inv3-*` files plus `local_rules_final.xml` on wazuh-server (listed below), covering the baseline login, threshold timeline, FIM events, rule 100040 and 100041 alerts, the 10:03 missed-event window and the 8 Oct reboot test (Figure 19).

```
Last login: Thu Oct  8 07:42:53 2026 from 192.168.56.1
senamile_n@wazuh-server:~$ ls -la ~/evidence-inv3-* ~/local_rules_final.xml
-rw-rw-r-- 1 senamile_n senamile_n  114 Oct  2 07:28 /home/senamile_n/evidence-i
nv3-5763-escalation.tsv
-rw-r--r-- 1 root       root       1006 Oct  1 10:07 /home/senamile_n/evidence-i
nv3-baseline-normal-login-senamile.json
-rw-rw-r-- 1 senamile_n senamile_n  770 Oct  2 08:12 /home/senamile_n/evidence-i
nv3-fim-ransomware.tsv
-rw-rw-r-- 1 senamile_n senamile_n  998 Oct  2 07:22 /home/senamile_n/evidence-i
nv3-jdoe-login-5715.json
-rw-rw-r-- 1 senamile_n senamile_n  461 Oct  1 12:46 /home/senamile_n/evidence-i
nv3-manager-restarts.txt
-rw-rw-r-- 1 senamile_n senamile_n  938 Oct  2 07:22 /home/senamile_n/evidence-i
nv3-miss-window.tsv
-rw-rw-r-- 1 senamile_n senamile_n  580 Oct  2 08:12 /home/senamile_n/evidence-i
nv3-ransomware-commands.tsv
-rw-rw-r-- 1 senamile_n senamile_n  294 Oct  8 07:56 /home/senamile_n/evidence-i
nv3-reboot-test.tsv
-rw-rw-r-- 1 senamile_n senamile_n  299 Oct  2 08:08 /home/senamile_n/evidence-i
nv3-rule-100040.json
-rw-rw-r-- 1 senamile_n senamile_n  137 Oct  2 09:13 /home/senamile_n/evidence-i
nv3-rule-100041.json
-rw-rw-r-- 1 senamile_n senamile_n  251 Oct  1 12:32 /home/senamile_n/evidence-i
nv3-success-after-failure.json
-rw-rw-r-- 1 senamile_n senamile_n  543 Oct  2 07:29 /home/senamile_n/evidence-i
nv3-threshold-timeline.tsv
-rw-r----- 1 senamile_n senamile_n 1787 Oct  2 09:22 /home/senamile_n/local_rule
s_final.xml
senamile_n@wazuh-server:~$
```

Figure 23. *Evidence inventory on wazuh-server:* `ls -la ~/evidence-inv3-* ~/local_rules_final.xml`.

7. CONCLUSION

Outcome, lessons learned, action plan and sign-off for both teams

7.1 Outcome against the exercise objectives

Objective	Result	Evidence
Detect SSH brute force and the compromised login	Met	Rules 5763 (L10) and 40112 (L12) fired on 1 Oct; 84 s to escalate
Detect discovery on the sensitive folder	Met	Custom rule 100040 (L9) fired on every audited command touching company_docs
Detect ransomware-style impact in real time	Met	Custom rule 100041 (L12) fired 43 ms after the first delete; attack lasted 64 ms
Recover without data loss	Met	4 of 4 files restored with original names, sizes and timestamps

Detect every login after repeated failures	**Partly met**	Three of six successful logins never reached Wazuh before the fix; none lost since (finding 6.3)
Alert only on malicious activity	**Partly met**	The administrator restore raised a false positive at 09:22:19.832 (finding 6.1)

7.2 Lessons learned

- **What worked:** realtime FIM, custom rules built from MITRE techniques, and consistent timestamps across auditd, FIM and sshd, which made the 64 ms timeline possible.
- **Speed is the real constraint:** detection at +43 ms is not useful for containment unless the response is automatic.
- **Default thresholds are tuned for volume, not for a careful attacker:** 7 failures followed by a success can pass at level 5.
- **Detections need a test for false positives:** a rule that works against an attack must also be run against the matching administrator action.
- **Logging can fail silently:** a missing alert is only found by comparing Wazuh against the source log, as done here with journalctl.
- **Separate what is proved from what is believed:** the journald explanation fits every case I have seen, but only a controlled test would prove it. Writing down which statements are evidence and which are inference showed me exactly which test to run next.

7.3 Action plan

#	Action	Finding	Priority	Timeframe
1	Enable Wazuh active response on rule 5763 (block the source IP) and a custom account-lock response on 100041	6.4	High	Immediate
2	Reset the jdoe password, enforce SSH keys, add lockout or rate limiting	6.5	High	Immediate
3	Tune rule 100041 to match deletes paired with `.locked` adds, then re-test against the restore	6.1	High	Short term
4	Add a custom rule for 5 failures and for any success after 3+ failures from one IP	6.2	Medium	Short term
5	Done: added /var/log/auth.log as a second SSH log source (2 Oct); a reboot test on 8 Oct confirmed it delivered both logins that journald lost. Remaining: alert on logcollector rotation messages and re-run the 10:03 test	6.3	Medium	Short term
6	Repeat the full red team chain after the changes and compare results with section 4.4	All	Medium	Next lab cycle

7.4 Closing statement

The exercise showed that Wazuh with auditd and realtime file integrity monitoring detects each stage of a credential-to-ransomware attack at least once, and that two custom rules close gaps in the default ruleset. Some individual successful logins were lost before analysis (finding 6.3). The incident was fully recovered, and the blue team response followed all four phases of the lifecycle. The remaining risk is in speed, tuning and collection reliability: containment must become automatic, rules 100041 and 5763 need adjusting, and SSH log collection must be monitored so any silent loss is noticed. Completing the action plan above and repeating the attack will confirm that the gaps are closed.

Prepared by	Role	Date
Senamile N Maphoso	SOC analyst (blue team) and red team operator	8 October 2026